

Solving Combinatorial Optimization Problem over Graph through QUBO Transformation and Deep Reinforcement Learning

Tianle Pu^{1,*}, Chao Chen^{1,*}, Li Zeng^{1,*}, Shixuan Liu¹, Rui Sun¹, Changjun Fan^{1,†}

¹ *Laboratory for Big Data and Decision*

College of Systems Engineering

National University of Defense Technology

Changsha, China

{ putl22, chenc1997, zengli24, liushixuan, sunr23, fanchangjun }@nudt.edu.cn

Abstract—Many complex problems encountered in both production and daily life can be conceptualized as combinatorial optimization problems (COPs). Many ad-hoc deep learning methods have been proposed to solve these problems, but there still lacks an effective unified framework. In this work, we take a step towards this goal by designing an unified end-to-end deep reinforcement learning framework named CONQUER. CONQUER first translates various COPs into the unified QUBO (Quadratic Unconstrained Binary Optimization) formalization, and afterwards trains a neural QUBO solver to search high-quality strategies, which are then checked and repaired to obtain the solutions for original COPs. The QUBO solver is a purely data-driven neural model, which does not rely on any expert inputs. We adopt the graph transformer network to represent the QUBO graph, so as to effectively capture the features of problem objective and constraints. The Deep Q-network is utilized to train the QUBO solver to find long-sighted solution strategies. Despite being trained solely on small synthetic graphs, CONQUER exhibits promising generality on much larger instances, and can be easily adapted to real-world datasets. Experimentally, we show that CONQUER achieves the state-of-the-art performances on four COPs over graphs, including minimum vertex cover, maximum independent set, maximum clique and maximum cut problem.

Index Terms—Combinatorial Optimization, Quadratic Unconstrained Binary Optimization, Reinforcement Learning, Graph Transformer

I. INTRODUCTION

Combinatorial optimization problems (COPs) are ubiquitous in real-world applications, spanning from industry [1], transportation [2] to military planning [3]. Generally, most problems relate to making choices in scenarios where numerous binary decisions must be made, with each set of choices resulting in a corresponding objective score. Furthermore, most of COPs are inherently NP-hard, making them challenging to solve and attracting a great of interest from the fields of optimization and computer science. Specifically, most COPs are decided directly on the graph, such as the Maximum Cut problem [4] and the Minimum Vertex Cover problem [5]. Moreover, it is natural to use a bipartite graph to represent the interactions

between variables and constraints within COPs. And these graphs exhibit rich problem information and common patterns.

As a matured research topic for several decades, at least three primary strategies have emerged: exact algorithms [6], approximate algorithms [7], [8], and heuristic algorithms [9]. Exact algorithms could obtain the global optima but are prohibitive for large instances. Approximate algorithms have theoretical gap guarantee and sometimes polynomial time complexity, but suffer from difficult construction and weak empirical performances. Heuristic algorithms are fast and often satisfactory in practice, but are criticised for their lack of theoretical guarantees and need to be tailored to each specific problem.

However, there are at least two key challenges for traditional methods. First, **most of COPs are intrinsically NP-hard, which makes them very difficulty to solve.** Existing strategies often face the so-called trade-off dilemma: fast algorithms fail to ensure solution quality, whereas solutions of higher quality often takes longer time. Second, **most methods develop algorithms customized to the mathematical structure of specific problems, which limits their applicability beyond the original intent.** Despite much success achieved, most of them have severe reliance on particular structure of the problem. And the diverse nature of application in practice often calls for a variety of creation techniques, each of which has limited applicability beyond its original intended purpose.

Here, a natural question arises, *can we have a model that is able to efficiently solve a wide range COPs?* In this work, we propose CONQUER (solving Combinatorial Optimization problem over graph through QUbo transformation and dEEP Reinforcement learning), providing a potential practical choice towards this goal. Specifically, CONQUER tackles the above two mentioned challenges as follows.

In terms of the ad-hoc challenge, CONQUER adopts the QUBO model as the “translator” for various COPs. QUBO, an acronym for a Quadratic Unconstrained Binary Optimization, is a mathematical formulation that could describe a vast variety of important COPs found in industry, science and other domains, as detailed in previous work [10], [11]. Through

* These authors contributed equally to this work. † Corresponding author.

special reformulation techniques that are not hard to adopt, many important COPs can be solved with a QUBO solver once they are described with the QUBO language.

In terms of the trade-off challenge, CONQUER develops a deep learning based QUBO solver, which treats the QUBO-solving process as a learning task. The goal is to train a better heuristic to minimize the objective value for the QUBO problem through a vast number of problem instances from a specific distribution, such that it can effectively handle new, unseen instances from the same distribution. Once these neural models are well trained, they can be efficiently applied multiple times and solve numerous COPs.

In general, CONQUER is developed in an end-to-end manner, mainly have the following key steps. It begins by transforming different COPs with the QUBO language and obtaining the corresponding QUBO graph. Subsequently, these graphs are represented with a graph transformer network. Then we use reinforcement learning to develop a better heuristic to achieve a minimal objective value for the QUBO problem. And finally, it verifies and corrects the solutions to ensure their validity for the original problem. The most significant advantage of our work compared to previous work is that we do not need to adjust the model architecture for any COPs, which dramatically reduces the cost of problem customization.

We conduct extensive experiments on four COPs over graph, including minimum vertex cover, maximum independent set, maximum clique and max-cut problem. For each problem, we test CONQUER on both synthetic (with varying sizes) and real-world data (from different domains). The results show CONQUER can effectively solve four COPs, surpassing most state-of-the-art (SOTA) competitors, including both traditional heuristic methods and learning-based methods. In some cases, CONQUER even achieves the same accuracy as the modern commercial solvers.

Our main contributions are summarized as follows:

- 1) To the best of our knowledge, CONQUER is a step towards developing a neural solver tool that goes beyond a mere methodological framework, unifying diverse modeling approaches and reducing the customization costs associated with problem modeling.
- 2) We design an end-to-end deep reinforcement learning architecture that uses QUBO language as its modeling language, unifying the objectives and reward function designs for the graph optimization problems under QUBO formulations. By employing a consistent model architecture to tackle diverse problems, CONQUER broadens the generalization boundaries of traditional neural solvers, enabling them to achieve cross-task generalization.
- 3) We perform extensive experiments on four COPs, with each we test on both synthetic and real-world datasets in different scales and in different domains. Our results demonstrate that CONQUER performs better than state-of-the-art competitors, and in some cases, even achieves the same accuracy as the commercial solvers.

II. RELATED WORKS

In recent years, many researchers pay much attention to the problem-specific characteristics to improve the accuracy of the COPs, and have made great achievements. They often treat finding the optimal solution as a decision-making process, and employ some data-driven methods to find heuristics. Among them, deep reinforcement learning, driven by data, allows the agent to continuously interact and learn from trial and error with the environment, yielding promising results. S2V-DQN [12] pioneers this solving paradigm by embedding the state of the graph and using Q-learning to maximize the objective function of COPs. Subsequently, ECO-DQN [13] addresses the issue of insufficient exploration and improves the exploration efficiency. Moreover, meta-learning [14] and sampling-based methods [15]–[18] are also gradually coming into focus. These methods achieve success by leveraging the unique problem structure and through the careful design of targeted features. However, this reliance on problem-specific characteristics poses a challenge when it comes to transferring to other problems, greatly increasing the cost of customization for each new problem.

Recent progress has shown the strong power of the QUBO model in expressing many COPs, thanks to the unified QUBO formulation [19]. Notably, PI-GNN [20] innovates the integration of QUBO model with graph neural networks (GNN), paving new paths for addressing COPs. They employ a relaxation technique to create a differentiable loss function for training GNNs, and after the unsupervised learning phase, it performs a simple mapping for integer variables. Building on the above work, [21] extends the method to tackle graph coloring problems. In addition, GRL [22] also attempts to solve a diverse array of COPs from the perspective of line graphs. GRL transforms complex edge-driven COPs into vertex-driven ones through line graph conversion, and then constructs the feasible solution by adopting a point-by-point selection strategy. Unfortunately, previous work only proposed a methodology framework, whereas ours put forward a general COP neural solving tool. For example, GRL formulates all different COPs as the same MDP process, but it still requires the design of different reinforcement learning components for each problem. Furthermore, Our work broadens the generalization capabilities of existing solving methods, enabling generalization across similar problems. The magic behind is that all COPs could be translated with QUBO formulations, once we have a powerful QUBO solver, we can then solve all COPs. The mechanism behaves like the commercial solver Gurobi [23] or CPLEX [24], which solves different COPs after translating them into the MIP formulations. Gurobi or CPLEX is just a MIP solver, but our CONQUER is essentially a universal neural QUBO solver tool.

III. PRELIMINARIES

A. Combinatorial Optimization Problem

Most COPs are inherently represented on graphs, and those that aren't can often be formulated as bipartite graphs, which

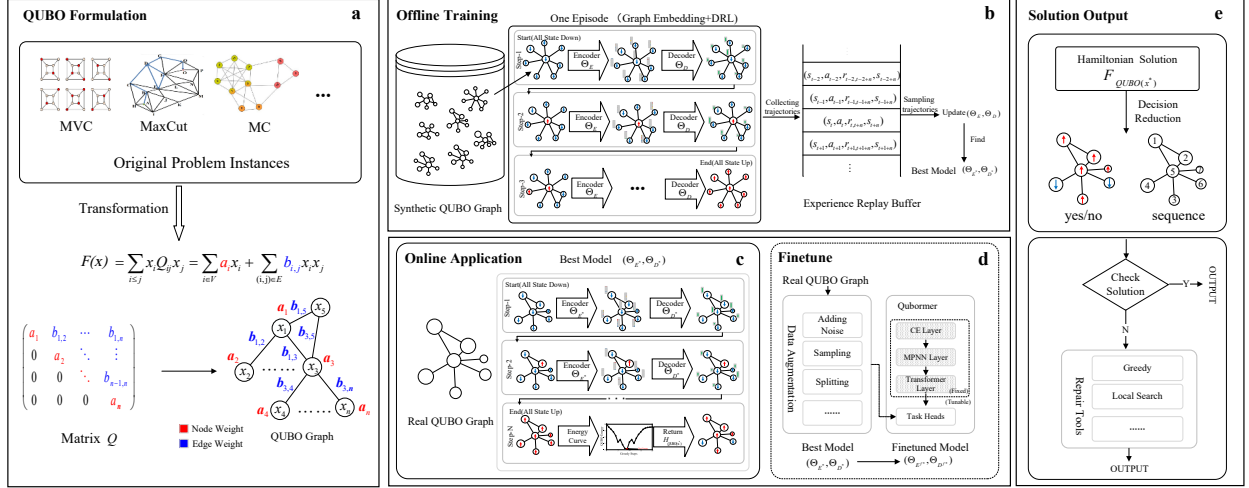


Fig. 1. The Workflow of CONQUER. It mainly consists of the following key steps: (a) Problem Transformation: transform the original problem and construct the corresponding QUBO graphs. (b) Offline Training: represent the QUBO graph with a graph transformer network and find the heuristic which results in a minimal objective within the QUBO problem through reinforcement learning. (c) Online Application: solve the QUBO problem through the well-trained QUBO solver. (d) Finetune (Optional): prepare the finetune dataset through data augmentation and obtain the finetune model. (e) Solution Output: decision reduction, checking, and repairing when the infeasible solutions appear.

capture the interactions between variables and constraints. COPs represented on graphs are typically decision-making problems that can be modeled as binary problems. The mathematical formulation for these problems can be described as follows:

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & h(x) \leq 0 \\ & g(x) = 0 \\ & x \in \{0, 1\} \end{aligned} \quad (1)$$

where $h(\cdot)$ indicates inequality constraints and $g(\cdot)$ indicates equality constraints.

To transform Eq. 1 into an unconstrained one, we can add penalty terms into the objective function for violating the constraints. Specifically, Eq. 1 can be easily rewritten as follows:

$$\min \quad f(x) + \lambda_1 \max[h(x), 0] + \lambda_2 \|g(x)\|^2, \quad x \in \{0, 1\} \quad (2)$$

where λ_1, λ_2 are penalty terms for inequality and equality constraints and $\|\cdot\|$ denotes the norm.

B. QUBO Formulation of CO

The QUBO serves as the cost function for a binary COP, incorporating both linear and quadratic terms. Specifically, the QUBO problem can be formalized as the following expression:

$$F(x) = \mathbf{x}^T Q \mathbf{x} = \sum_{i,j} x_i Q_{ij} x_j = \sum_{i=1}^n (a_i x_i + \sum_{i < j} b_{ij} x_i x_j) \quad (3)$$

where $\mathbf{x} = (x_1, x_2, \dots, x_n)$ represents a decision vector composed of binary variables $\{0, 1\}$, the matrix Q is an upper triangular matrix and $a_i = Q_{ii}$, given that $x_i^2 = x_i$ and $b_{ij} = Q_{ij}$.

IV. METHODOLOGY

This section outlines the workflow of CONQUER, as shown in Fig. 1. It begins by transforming the original COP into a QUBO graph. During the offline training phase, deep reinforcement learning is employed to obtain the optimal QUBO solver. Subsequently, in the online application phase, the well-trained QUBO solver is used to solve the QUBO problem. For out-of-distribution data, an efficient strategy involving data augmentation is provided to fine-tune the model. Finally, the output solution includes steps for decision reduction, verification, and repair of any infeasible solutions.

A. Problem Transformation and QUBO Graph Construction

TABLE I
QUBO FORMULATIONS OF COPs

Problem Type	QUBO Formulation
MVC	$F(x) = \sum_{i \in V} x_i + p \sum_{(i,j) \in E} (1 - x_i)(1 - x_j)$
MIS	$F(x) = - \sum_{i \in V} x_i + p \sum_{(i,j) \in E} x_i x_j$
MC	$F(x) = - \sum_{i \in V} x_i + p \sum_{(i,j) \notin E} x_i x_j$
MaxCut	$F(x) = - \sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j)$

According to Eq. 3, we can define a corresponding undirected graph $G = (V, E)$, node $v_i \in V$ represents the decision variable x_i in Eq. 3, with a weight equal to the coefficient of the linear term a_i . Coupled nodes v_i and v_j , connected by an undirected edge $e_{ij} \in E$, have a weight equal to the coefficient of the quadratic term b_{ij} in Eq. 3. Compared with previous work conducted directly in different original problem spaces, a significant advantage of this graph representation is

its ability to encode the objective and constraint information of the optimization problem into the graph structure, which allows the neural network to learn directly from the graph. The QUBO formulation of the COPs studied in this work is provided in Table I. In this formulation, $x_i \in \{0, 1\}$ represents a binary variable, and p denotes the penalty term.

B. Graph Representation with Qubormer

Intuitively, the QUBO graph encapsulates rich information, including the objective and constraints of the COP. Therefore, our goal is to discover an embedding vector that effectively captures the attributes of each node (variables) while also representing the interactions among both adjacent and long-range nodes (constraints). Given the unique structure of the QUBO graph, we have designed a novel Graph Transformer Network named Qubormer. This network primarily consists of two components: an encoder and a decoder.

1) *Encoder*: Recent studies [25]–[27] have highlighted that traditional message-passing networks face bottlenecks such as over-smoothing, making them less effective for tasks requiring long-term dependencies. To address these limitations, we propose integrating the transformer mechanism [28]–[30] into message-passing networks. Broadly, the encoder consists of three key components.

Centrality Encoding. By integrating Centrality Encoding, we can capture the significance of nodes within the graph. Each node is enhanced with its degree, providing a measure of its relative importance in the network topology.

$$\mathbf{h}_i^{(0)} = \text{ReLU}(\mathbf{W}_1 x_i) + \text{ReLU}(\mathbf{W}_2 z_{\text{deg}}(v_i)) \quad (4)$$

where x_i is the initial features of the node, $z_{\text{deg}}(v_i)$ encodes the degree of the node, $\mathbf{W}_1$ and $\mathbf{W}_2$ are learnable parameters. **Message-passing Layer.** To be precise, it is evident that both the features of the nodes and the edges in the QUBO graph are equally crucial, particularly because the edge features capture the interactions between nodes, which are intrinsically linked to the constraints. Therefore, we introduce a two-step neighborhood aggregation operation, which includes “node-to-edge” aggregation and “edge-to-node” aggregation, to represent both nodes and edges simultaneously.

$$\mathbf{h}_{M_{ij}}^{(l)} = \text{ReLU}([\mathbf{W}_4(h_{M_i}^{(l-1)} + \mathbf{h}_{M_i}^{(l-1)}), \mathbf{W}_5 \mathbf{h}_{M_{ij}}^{(0)}]) \quad (5)$$

$$\mathbf{h}_{M_i}^{(l)} = \text{ReLU}([\mathbf{W}_6 \sum_{v_j \in \mathcal{N}(v_i)} \mathbf{h}_{M_{ij}}^{(l)}, \mathbf{W}_7 \mathbf{h}_{M_i}^{(l-1)}]) \quad (6)$$

where $\mathbf{h}_{ij}^{(0)} = \text{ReLU}(\mathbf{W}_3 x_{ij})$ is initial edge embedding vector and x_{ij} is the edge input feature. $\mathbf{W}_3, \mathbf{W}_4, \dots, \mathbf{W}_7$ are all learnable parameters. $[\cdot]$ denotes concatenation. Furthermore, to mitigate the issue of over-smoothing, we design a residual connection mechanism within the consecutive processes of node and edge feature updates, such that:

$$\mathbf{h}_{M_i}^{(l)} = \mathbf{W}_8 [\mathbf{h}_{M_i}^{(l)}, \mathbf{h}_{M_i}^{(l-1)}] \quad (7)$$

where $\mathbf{W}_8$ is learnable parameter. $[\cdot]$ denotes concatenation.

Transformer Layer. In spite of the great progress attained, traditional methods often emphasize directly stacking Transformer layers on top of several layers of the message-passing network [31]. Although these methods can achieve richer representations of nodes and edges, they still struggle with the core limitation of over-smoothing. Hence, we present a fresh perspective: within each layer, we aggregate the output of the message-passing network with that of the global attention layer to update node features collaboratively.

$$\mathbf{h}_{T_i}^{(l)'} = \text{MHA}(\text{LN}(\mathbf{h}_{M_i}^{(l-1)})) + \mathbf{h}_{M_i}^{(l-1)} \quad (8)$$

$$\mathbf{h}_{T_i}^{(l)'} = \text{FFN}(\text{LN}(\mathbf{h}_{T_i}^{(l)'})) + \mathbf{h}_{T_i}^{(l)'} \quad (9)$$

where MHA denotes the Multi-Head Attention network, LN represents the Layer Normalization, and FFN signifies the Feed-Forward Neural Network layer.

Finally, after T layers of iterative computation, we can obtain the final representation $\mathbf{z}_i = \mathbf{h}_i^{(T)}$. Furthermore, to represent the entire QUBO graph, we aggregate the embeddings of all nodes by summing them, i.e., $\mathbf{z}_s = \sum_{i \in V} \mathbf{z}_i$.

2) *Decoder*: Before going into the details about decoders, it is necessary to recall the Markov Decision Process (MDP) used for solving QUBO problems. Formally, we consider an MDP described by the tuple (S, A, P, R) , where S is the set of all possible states $s_i \in S$, with s_i representing the the real-time state of QUBO graph G_i at step i . An action $a_i \in A$ refers to the assignment of binary variables $x_i \in \{0, 1\}$ for the QUBO problem instance given state s_i . The policy $\pi : S \rightarrow A \in P$ denotes the heuristic found by CONQUER. The reward $r_i = a_i + \sum_{(i,j) \in E} b_{ij} x_j \in R$, obtained by executing action a_i in state s_i , denotes the change in the objective function value before and after the action, as calculated by Eq. 3. From this, we can see that the greatest appeal of CONQUER is that it shares the entire MDP, not just the methodological aspects, which makes it easy for CONQUER to generalize across different problems.

The decoder essentially transforms a pair of state-action pairs (s_i, a_i) into a scalar $Q(s_i, a_i; \Theta)$, which evaluates the quality of action a_i in state s_i . The function Q can be succinctly represented as follows:

$$Q(\mathbf{h}, v; \Theta) = \mathbf{W}_{11} \text{ReLU}(\mathbf{W}_{10} \text{ReLU}(\mathbf{W}_9 [\mathbf{z}_s, \mathbf{z}_i])) \quad (10)$$

where $\mathbf{W}_9, \mathbf{W}_{10}, \mathbf{W}_{11}$ are learnable parameters. $[\cdot]$ denotes concatenation. Additionally, $\Theta = \{\mathbf{W}_k\}_{k=1}^{11}$ represents the parameters of both the encoder and decoder.

C. Reinforcement Learning for QUBO Optimization

1) *Training*: We employ the extensively applied DQN [32] for training. CONQUER first generates different instances of the original problem for preparation and constructs the corresponding QUBO graphs. Then, CONQUER will play an entire “game” on the QUBO graph to obtain a variety of trajectories and cut them into several transitions with an internal step. Finally, CONQUER selects transitions for learning and updates the network parameters accordingly.

Data Preparation. In fact, the QUBO formulation serves as a “converter”, which bridges different COPs into the unified QUBO graphs. This “converter” can be easily implemented according to Table I. It is worth noting that in this work, we separately train independent CONQUER for various COPs, adhering to the principle of independent and identically distributed (I.I.D.) data.

Trajectory Generation. As described in Sec. IV-B, we formulate the problem-solving process of QUBO as an MDP. Next, we will introduce some terms to better understand the training process. An *episode* is a complete optimization process on the QUBO graph to find the best variable assignments that result in the minimal objective of the QUBO, requiring all variables to be flipped from 0 to 1 sequentially. The *trajectory* generated from an episode is a state-action-reward sequence $\{s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T\}$, where s_0 is the initial state with all variables set to 0, and s_T is the terminal state with all variables set to 1. Playing a single “game” entails completing a full episode on the QUBO graph, which includes generating trajectories and collecting experience transitions $s_t, a_t, r_{t,t+n}, s_{t+n}$, where $r_{t+n} = \sum_{k=0}^{n-1} \gamma^k r(s_{t+k}, a_{t+k}, s_{t+k+1})$, and storing them in the replay buffer $\mathcal{B}$. Here, the parameters n for delayed rewards and γ for discount factor are hyper-parameters.

Update Network Parameters. CONQUER selects transitions from the replay buffer to learn and update the network parameters Θ in order to minimize the following loss function.

$$\mathcal{L} = \mathbb{E}[(r_{t,t+n} + \gamma \max_{a_{t+n}} Q(s_{t+n}, a_{t+n}; \hat{\Theta}) - Q(s_t, a_t; \Theta))^2] \quad (11)$$

where $(s_t, a_t, r_{t,t+n}, s_{t+n}) \sim \mathcal{B}$ denotes sampling n step state-action-reward sequences from the replay buffer, $\hat{\Theta}$ refers to the parameters of the target network, typically updated with the parameters Θ periodically.

2) *Online Application and Fine-tune:* In practice, the Q -greedy strategy iteratively flips the state of each variable in the QUBO graph from 0 to 1. During this process, the assignment of variables that results in the lowest objective value is identified as the optimal solution for the QUBO problem. However, in the field of machine learning, out-of-distribution generalization has long been a prominent challenge, and CONQUER is no exception. To tackle this challenge, we propose a concise and efficient fine-tuning approach. Firstly, we perform data augmentation on the fine-tuning dataset, such as introducing noise to the QUBO graph, partitioning the dataset into train/validation/test subsets, and conducting sampling on the real-world graph. Then, we load the pre-trained model and freeze its Encoder part, only fine-tune the task head for downstream tasks. Upon completing the fine-tuning process, the fine-tuned model undergoes testing, mirroring the procedures of online application.

D. Solution Output

1) *Decision Reduction:* In fact, the solutions obtained directly from the model are solutions to the QUBO problem, representing the values of variables that minimize the objective. To obtain solutions to the original COP, a process of decision reduction is required. Specifically, this can be broadly

categorized into two main types. The first type involves “yes/no” decision outputs, used to determine if variables are included in the solution set. The second type encompasses sequential decision outputs, which are crucial for determining both whether variables are included in the solution set and the order of these variables within the solution set.

2) *Check and Repair Solution:* If the model fails to train sufficiently due to sparse data or insufficient training time, or if the penalty term is set too low, it may result in the neural network finding solutions that do not meet the feasibility conditions of the original problem. To mitigate above issues, it is essential to first ensure the legality of the solutions. If a solution adheres to the feasibility conditions of the original problem, it is directly outputted; if not, it needs to be repaired to meet these conditions. Generally, the repair methods are simple and inexpensive, often employing common greedy techniques. This step is usually optional and is only used on a few large-scale real datasets in this work.

V. EXPERIMENTS

In this section, we demonstrate the efficiency and flexibility of CONQUER through experiments on four COPs: minimum vertex cover (MVC) [5], maximum independent set (MIS) [33], maximum clique (MC) [34], and maximum cut (MaxCut) [13]. Our experimental computational environment is based on the Tesla V100 GPU, where we perform experiments on both synthetic and real-world datasets. Furthermore, we compare our results with the SOTA ML-based methods, heuristic methods, and commercial solvers.

A. Experimental Setup

1) *Baseline methods:* For MaxCut problems, we use semi-definite programming (SDP) [35] to obtain the relaxation solution and adopt two heuristic methods: greedy [36]–[38] and mean-field annealing (MFA) [39]. For comparison, we implement classical S2V-DQN [40] and two probabilistic view-based methods, EGN [41] and ANNEAL [42], and evaluate against the current SOTA method, GFlowNet [15], which samples from the solution space. For non-MaxCut tasks, the variations in ML-based methods differ; we adopt unsupervised methods such as RUN-CSP [43] and EGN [41], along with its enhanced version, Meta-EGN [14]. Additionally, we compare with the SOTA ML-based method, VAG-CO [16], which enhances the MFA algorithm. Despite formulating different COPs into the unified Ising model which is similar to QUBO model, its primary contribution lies in refining and accelerating the annealing process.

2) *Datasets:* Recent research [12] has observed that the existing synthetic graph datasets present simplicity for certain COPs such as MaxCut. Hence, we select the RB graphs [44], known for their increased complexity, to conduct experiments on the MaxCut problem. For the other problems, we adopt the BA graphs [45]. For realistic data, our experiments into MIS, MC, and MVC employ graph datasets [46] spanning various domains, including biology and social networks, such as ENZYMES, PROTEINS, IMDB-BINARY, COLLAB, and

MUTAG, or selected subsets thereof. For the reset problem, namely MaxCut, we draw upon graph datasets sourced from the Opticom project¹, which have gained prominence in previous works.

3) *Evaluation metrics*: To be consistent with previous work, we employ a comprehensive evaluation metric that considers the best ratio AR^* and average ratio $\hat{AR}$ of the results obtained from each baseline method to those achieved by the Gurobi [23] within one hour, along with the corresponding standard error.

4) *Hyperparameter Setting*: CONQUER shares the same model architecture across all experimental settings, as we transform different problems into the unified QUBO formulation. The majority of our hyperparameter settings parallel those of S2V-DQN, and we have made no adjustments to them. We are confident that if an exhaustive search for the optimal hyperparameters were conducted, the performance of CONQUER would be further improved. The detailed hyperparameter settings of CONQUER are shown in Table II.

TABLE II
HYPERPARAMETER SETTINGS FOR CONQUER.

Name	Value	Description
M	5×10^5	Replay Memory Size
lr	1×10^{-4}	Learning Rate
d	64	Embedding Dimension
C	10^3	Update Time
T	5	Layer Iterations
Ω	10^6	Maximum Episodes
γ	1	Discount Factor
n	3	Q-learning Steps
B	64	Batch Size
h	4	Transformer Heads
p	1.2	Penalty Terms

B. Results on synthetic data

1) *Evaluation on Synthetic Data for MVC*: The MVC problem involves finding the smallest set of nodes in a graph such that every edge in the graph is connected to at least one node in this set. Following the same setup as in [40], CONQUER is trained on BA graphs with node sizes ranging from 15 to 20 nodes. Then, the well-trained model is tested on BA graphs with node sizes of 15-20, 40-50, 50-100, 100-200, 200-300, and 400-500 nodes. Fifty problem instances are tested for each node size, recording the average ratio $\hat{AR}$ and standard deviation, as shown in Fig. 2. It is evident that CONQUER achieves precision almost on par with commercial solvers for instances with fewer than 500 nodes, significantly outperforming greedy methods. Compared to the currently high-performing machine learning algorithm S2V-DQN, CONQUER shows noticeable improvements across all problem sizes. Notably, CONQUER achieves approximately a 2.99% performance improvement over S2V-DQN, which already performs close to commercial solvers.

¹<https://grafo.etsii.urjc.es/opticom>

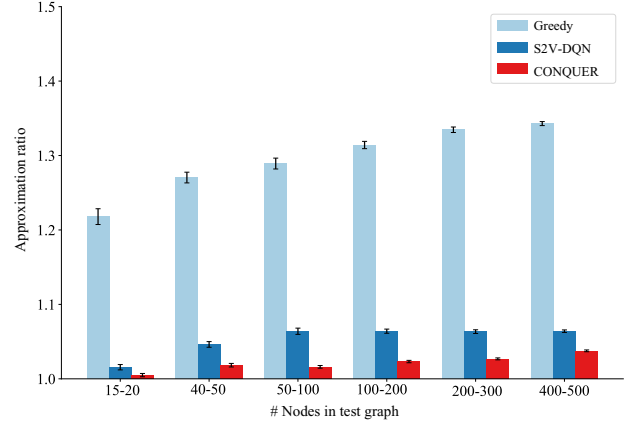


Fig. 2. Results on Synthetic Data for MVC.

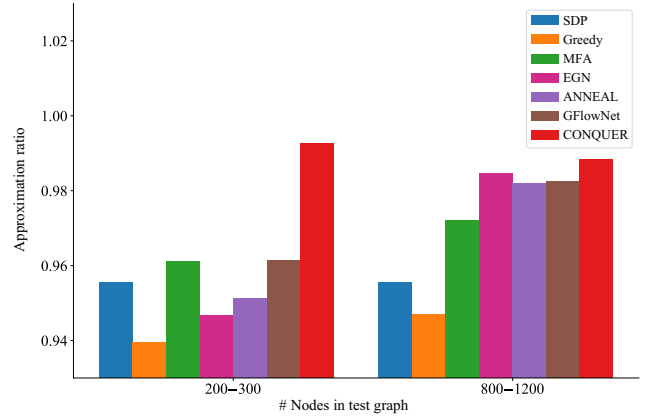


Fig. 3. Results on Synthetic Data for MaxCut.

2) *Evaluation on Synthetic Data for MaxCut*: The MaxCut problem involves finding a cut in a graph such that the sum of the weights of the cut edges is maximized. For the MaxCut problem, we use relatively complex RB graphs for MaxCut problem relatively complex RB graphs in which parameter p denotes the difficulty of the graph. According to the number of nodes, the graphs are categorized into three scales: “very small”, “small”, and “large”. The “very small” scale consists of nodes numbering 25-100, the “small” scale has nodes numbering 200-300, and the “large” scale includes nodes numbering 800-1200. To ensure a diverse range of difficulty levels, the p for “small” scale problem instances are set between 0.3 and 1, while for the “very small” and “large” scales, the p are set between 0.25 and 1. CONQUER is trained on the “very small” scale and tested on 50 problem instances of the other two scales, recording the respective means and standard deviations, as shown in Fig. 3. It is clear that the model trained on the “very small” scale can still outperform the existing SOTA ML-based methods, on “large” scale, and achieve performance close to that of commercial solvers with

TABLE III
SYNTHETIC RESULTS FOR MIS AND MC.

MIS				
	50-100	100-200	200-300	400-500
Greedy	0.9218 $\pm$ 0.0060	0.9315 $\pm$ 0.0040	0.9254 $\pm$ 0.0033	0.9264 $\pm$ 0.0021
CONQUER	0.9930 $\pm$ 0.0021	0.9861 $\pm$ 0.0020	0.9863 $\pm$ 0.0017	0.9804 $\pm$ 0.0013
MC				
	50-100	100-200	200-300	400-500
Greedy	0.9410 $\pm$ 0.0138	0.9300 $\pm$ 0.0151	0.9200 $\pm$ 0.0168	0.9050 $\pm$ 0.0195
CONQUER	0.9460 $\pm$ 0.0136	0.9390 $\pm$ 0.0139	0.9280 $\pm$ 0.0165	0.9114 $\pm$ 0.0191

over 99% accuracy on “small” scale.

3) *Evaluation on Synthetic Data for MIS and MC*: These two cases show that CONQUER leads to significant performance benefits when applied to the synthetic data. Furthermore, we conduct experiments on synthetic data for MIS and MC problems. We train models on BA graphs with node scales ranging from 15 to 20, and then test up to graphs with node scales of 400-500. Comparing the CONQUER with greedy methods across all experimental settings, we consistently achieve favorable results, as shown in Table III.

C. Results on real-world data

Moreover, we conduct more in-depth experiments to validate CONQUER on out-of-distribution real-world datasets to evaluate its efficiency, scalability, and generalization. We fine-tune the model using the strategy described in Sec. IV-C and denote the results as “f-t”. To highlight the experimental results, the optimal results are indicated in bold, while the sub-optimal results are underlined.

1) *Evaluation on Real-world Dataset for MVC*: We compare CONQUER with the results obtained by VAG-CO [16], which address the MVC problem on the COLLAB and IMDB-BINARY two datasets. Following VAG-CO, we partition the dataset into training, validation, and test sets at a ratio of 0.7/0.1/0.2, respectively, utilizing only the first 1000 graphs from the COLLAB. However, unlike VAG-CO, we employ the average approximation ratio $\hat{AR}$ as our evaluation metric. This is because, with the best approximation ratio AR^* , CONQUER can achieve the optimal solution across all datasets, which also applies to MIS and MC problems. The performance of each method on real-world datasets is compared in Table IV. Notably, CONQUER attains optimal results on the IMDB-BINARY. It only needs to retrain a few hundred steps to achieve optimal, while fine-tuning considerably enhances model accuracy on the COLLAB.

2) *Evaluation on Real-world Dataset for MIS and MC*: Similarly, for the MIS problem, we use five datasets: ENZYMES, PROTEINS, IMDB-BINARY, COLLAB, and MUTAG. These datasets were divided into training, validation, and test sets at a ratio of 0.6/0.1/0.3. For the COLLAB dataset, only the first 1000 graphs are used for experimentation. The CONQUER is trained on BA graphs with nodes ranging from

TABLE IV
RESULTS ON REAL-WORLD DATA ON MVC.

Method	COLLAB	IMDB-BINARY
Evaluation metric	AR^*	AR^*
EGN	1.013 $\pm$ 0.022	1.000
EGN f-t	1.008 $\pm$ 0.015	1.000
RUN-CSP	1.208 $\pm$ 0.198	1.188 $\pm$ 0.178
Meta-EGN	1.003 $\pm$ 0.010	1.000
Meta-EGN f-t	1.002 $\pm$ 0.010	1.000
Greedy	1.0636 $\pm$ 0.0037	1.0759 $\pm$ 0.0047
MFA	1.00021 $\pm$ 0.00003	1.000
MFA-Anneal	1.00023 $\pm$ 0.00004	1.000
VAG-CO	1.0017 $\pm$ 0.0002	1.000
CONQUER	1.000	1.000
Evaluation metric	$\hat{AR}$	$\hat{AR}$
MFA	1.00106$\pm$0.00001	1.0014 $\pm$ 0.0004
MFA-Anneal	1.0048 $\pm$ 0.0002	1.0011 $\pm$ 0.0002
VAG-CO	1.0056 $\pm$ 0.0002	1.0011 $\pm$ 0.0006
CONQUER	1.0172 $\pm$ 0.0021	1.0060 $\pm$ 0.0008
CONQUER-f-t	1.0065 $\pm$ 0.0010	1.000

15 to 20. During testing, two different models were employed: one directly trained on BA graphs and another fine-tuned separately for each dataset. Table V records the average ratio $\hat{AR}$, the best ratio AR^* , and their corresponding standard deviations for each method across 50 test problem instances. Results for other methods are sourced from VAG-CO [16]. It is evident that CONQUER achieves the performance of commercial solvers in terms of the best ratio AR^* across all datasets, and surpasses the current SOTA ML-based methods in terms of the average ratio $\hat{AR}$ on four out of the five datasets, excluding COLLAB.

Formally, the MC problem is similar to the MIS problem, as the MC problem can be conceptualized as solving the MIS problem on its complementary graph. CONQUER also shows superior performance when addressing MC problems. The experimental results for the MC problem are presented in Table VI. Analogous to the MIS problem, CONQUER consistently outperforms alternative approaches across all experiments.

3) *Evaluation on Real-world Dataset for MaxCut*: Following [40], we test ten graphs with edge weights $\{-1, 0, +1\}$ in the Optsicom project. To maintain experimental consis-

TABLE V
RESULTS ON REAL-WORLD DATA ON MIS.

Method	ENZYMES	PROTEINS	IMDB-BINARY	COLLAB	MUTAG
Evaluation metric	$\bar{AR}^*$	$\bar{AR}^*$	$\bar{AR}^*$	$\bar{AR}^*$	$\bar{AR}^*$
EGN	0.821±0.124	0.903±0.114	0.515±0.310	0.886±0.198	0.939±0.069
NSFE	0.775±0.155	0.729±0.205	0.679±0.287	0.392±0.287	0.854±0.132
REINFORCE	0.751±0.301	0.725±0.285	0.881±0.240	1.000	0.781±0.316
Straight-through	0.725±0.268	0.722±0.26	<u>0.917±0.253</u>	0.856±0.221	0.965±0.162
Greedy	0.9810±0.0009	0.9848±0.0003	1.000	0.9968±0.0001	<u>0.994±0.001</u>
MFA	0.9875±0.001	0.9883±0.0007	1.000	0.9985±0.0004	1.000
MFA-Anneal	0.9880±0.0014	0.9881±0.0007	1.000	0.9991±0.0004	1.000
VCG-CO	<u>0.9960±0.0007</u>	<u>0.9977±0.0005</u>	1.000	<u>0.9988±0.002</u>	1.000
CONQUER	1.000	1.000	1.000	1.000	1.000
Evaluation metric	$\bar{AR}$	$\bar{AR}$	$\bar{AR}$	$\bar{AR}$	$\bar{AR}$
MFA	0.9635±0.0014	0.9705±0.0012	0.990±0.002	0.960±0.007	0.970±0.003
MFA-Anneal	0.968±0.001	0.973±0.002	0.994±0.001	0.917±0.045	0.975±0.002
VCG-CO	0.9863±0.0007	0.9908±0.0005	0.995±0.001	0.993±0.001	0.994±0.002
CONQUER	0.9895±0.0021	0.9892±0.0020	<u>0.9963±0.0020</u>	0.9381±0.0064	0.9895±0.0021
CONQUER-f-t	0.9895±0.0021	0.9935±0.0014	1.000	<u>0.9859±0.0035</u>	1.000

TABLE VI
RESULTS ON REAL-WORLD DATA ON MC.

Method	ENZYMES	IMDB-BINARY
Evaluation metric	$\bar{AR}^*$	$\bar{AR}^*$
EGN	0.883 ± 0.156	0.936 ± 0.175
NSFE	0.933 ± 0.148	0.961 ± 0.143
REINFORCE	0.751 ± 0.301	0.881 ± 0.240
Straight-through	0.725 ± 0.268	0.917 ± 0.253
Greedy	0.9485±0.0034	0.9875±0.0008
MFA	0.9766 ± 0.001	0.999 ± 0.009
MFA-Anneal	<u>0.9921±0.0017</u>	<u>0.999±0.0001</u>
CONQUER	1.000	1.000
Evaluation metric	$\bar{AR}$	$\bar{AR}$
MFA	0.764 ± 0.013	0.983 ± 0.009
MFA-Anneal	0.806 ± 0.004	0.974 ± 0.004
VCG-CO	0.943 ± 0.012	0.993±0.0013
CONQUER	0.9445±0.0081	0.9814 ± 0.0046
CONQUER-f-t	0.9478±0.0078	0.9948±0.0039

TABLE VII
RESULTS ON MAXCUT PROBLEMS. BIGGER RESULTS INDICATE BETTER PERFORMANCE, WITH OPTIMAL RESULTS DISPLAYED IN BOLD AND SUB-OPTIMAL RESULTS SHOWN UNDERLINED.

Method	SDP	Greedy	S2V-DQN	CONQUER
Ratio	0.526	0.78	<u>0.9040±0.0040</u>	0.9711±0.0105

tency, we test both the re-implemented S2V-DQN and CONQUER directly on the real-world dataset. We conduct training on the BA graphs, and the results of each experiment are shown in Table VII.

D. Discussion

1) *Cross Task Inference*: CONQUER shares a formally consistent QUBO formulation to demonstrate significant advantages in the unified solving of COPs, thereby reducing the

customization costs associated with different problems. Given that both the training and testing phases of CONQUER are conducted using QUBO graphs, a model trained on one problem can be generalized to others due to the shared consistent model architecture. This section explores CONQUER's capability to generalize across different problems.

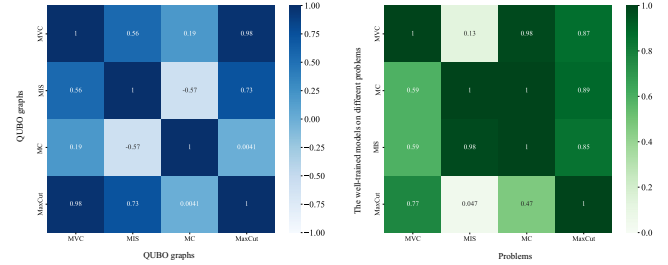


Fig. 4. Cross Task Performance. (a) the similarity between QUBO graphs of different problems. (b) the ability of CONQUER to generalize to different problems.

To evaluate this capability, four COP instances, including MVC, MIS, MC and MaxCut, are generated on BA graphs with nodes ranging from 50 to 100. These instances are then converted into QUBO formulations to obtain the corresponding QUBO matrices. The Pearson correlation between the QUBO matrices of these problems was calculated, as illustrated in Figure 4(a). Subsequently, separate CONQUER are trained on each of the four problems and tested on their respective instances. The average performance ratios across each problem are presented in Figure 4(b), where results closer to 1 indicate better performance. Analysis of these results yields several noteworthy conclusions. For instance, a model trained on the MIS problem exhibits excellent performance when solving the MC problem, which is expected given that the MC problem is equivalent to solving the MIS problem on the complement graph. Moreover, a model trained on the MVC

problem shows strong performance when solving the MaxCut and MC problems, but performs relatively poorly when applied to the MIS problem.

2) *Ablation Study for Qubormer*: The QUBO graph serves as a critical representation for capturing the objective functions and constraints of COPs. The effectiveness of GNNs in encoding these QUBO graphs is paramount, as it directly influences the quality of solutions to such problems. To illustrate this, we conduct a study focusing on the MVC problem using BA graphs with node sizes ranging from 15 to 20. In our experimental setup, we compare four models: the original Qubormer, a variant without centrality encoding (termed Qubormer-NoCE), another variant devoid of the Transformer layer (Qubormer-NoTransformer), and a traditional GCN for encoding purposes. During the testing phase, we scaled up the node sizes of the graphs to a range of 50 to 100 nodes. We then recorded and compared the mean ratios and standard deviations of the results.

The results, detailed in Fig. 5, align with our expectations. GCN demonstrates the poorest performance, which can be attributed to its inadequate representation of edge features. The centrality encoding layer, which emphasizes the significance of individual nodes within the QUBO graph, proves to be less critical, as its removal had the least impact on performance. On the other hand, the Transformer layer, which plays a vital role in mitigating the over-smoothing phenomenon typical in GNN models, is shown to be more influential. The removal of this layer had a more pronounced negative effect on performance compared to the removal of the centrality encoding layer.

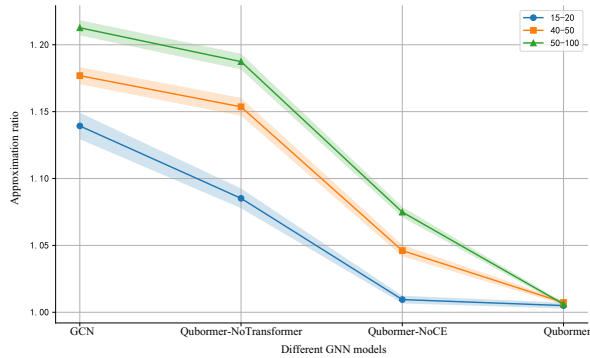


Fig. 5. Ablation Study for Qubormer.

3) *Different Penalty Terms*: When converting different COPs into QUBO formulations, a key challenge lies in handling constraints. The common approach involves introducing penalty terms into the objective function to account for the constraints. The magnitude of these penalty terms significantly impacts the learning process of the model. If the penalty term is too large, it may prevent reinforcement learning from converging. Conversely, if the penalty term is too small, it may lead to the model violating the constraints during the learning process, resulting in solutions that do not satisfy feasibility conditions.

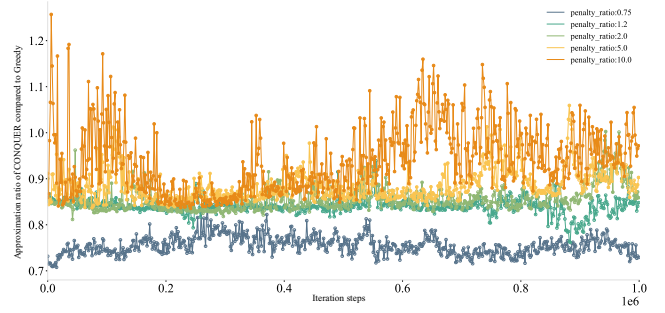


Fig. 6. Training Curves Showing the Impact of Varying Penalty Terms. Solid points indicate legal solutions, while empty points represent illegal solutions.

We experiment with various penalty values ranging from 0.75 to 10 for the MVC problem. CONQUER is trained on BA graphs with the node size ranging from 15 to 20, the different training curves under different penalty terms is shown in Fig. 6. When the penalty term is too low (below 1), the model struggles to find legal solutions, as indicated by the absence of solid points in the Fig. 6 for coefficients below 1, and predominantly empty points representing illegal solutions.

VI. CONCLUSION

This work presents the pioneering development of a unified end-to-end deep reinforcement learning architecture tailored for various COPs by formulating the problem into QUBO formulations. Due to the specific QUBO graph structure, we introduce the Transformer mechanism into the message-passing network to effectively capture the long-term dependencies between vertices. We then design corresponding decision-reduction strategies for different COPs alongside simple but efficient fine-tuning techniques for real-world datasets beyond the training distribution. Unlike all previous work, our framework stands apart by obviating the need for task-specific adjustments across various problem instances or fine-tuning scenarios.

Certainly, there are some limitations in our work. For example, the handling of COPs with complex constraints, such as routing problems, may not be as effective as hand-crafted heuristic methods. Currently, the conversion of COPs into QUBO formulation still relies on expert knowledge. In future research, it is possible to consider automating and efficiently converting this process using rule-based reasoning. However, we believe that our work offers a new perspective on creating a unified neural solver, with promising research prospects.

ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China (NSFC, 72421002, 72025405, 62206303), Science and Technology Innovation Program of Hunan Province (2023RC3009).

REFERENCES

- [1] G. Reinelt, "The traveling salesman: computational solutions for tsp applications," 1994. [Online]. Available: <https://api.semanticscholar.org/CorpusID:120717981>
- [2] P. Toth and D. Vigo, "The vehicle routing problem," 2001. [Online]. Available: <https://api.semanticscholar.org/CorpusID:209742615>
- [3] A. G. Kline, D. K. Ahner, and R. R. Hill, "The weapon-target assignment problem," *Comput. Oper. Res.*, vol. 105, pp. 226–236, 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:71142897>
- [4] F. Rendl, G. Rinaldi, and A. Wiegele, "Solving max-cut to optimality by intersecting semidefinite and polyhedral relaxations," *Mathematical Programming*, vol. 121, pp. 307–335, 2009. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16320923>
- [5] M. Ghaffari, T. Gouleakis, S. Mitrovic, and R. Rubinfeld, "Improved massively parallel computation algorithms for mis, matching, and vertex cover," *Proceedings of the 2018 ACM Symposium on Principles of Distributed Computing*, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:3478692>
- [6] E. L. Lawler and D. E. Wood, "Branch-and-bound methods: A survey," *Operations research*, vol. 14, no. 4, pp. 699–719, 1966.
- [7] D. P. Williamson and D. B. Shmoys, *The design of approximation algorithms*. Cambridge university press, 2011.
- [8] V. V. Vazirani, *Approximation algorithms*. Springer, 2001, vol. 1.
- [9] P. Festa, "A brief introduction to exact, approximation, and heuristic algorithms for solving hard combinatorial optimization problems," in *2014 16th International Conference on Transparent Optical Networks (ICTON)*. IEEE, 2014, pp. 1–20.
- [10] G. Kochenberger, J.-K. Hao, F. Glover, M. Lewis, Z. Lü, H. Wang, and Y. Wang, "The unconstrained binary quadratic programming problem: a survey," *Journal of combinatorial optimization*, vol. 28, pp. 58–81, 2014.
- [11] M. Anthony, E. Boros, Y. Crama, and A. Gruber, "Quadratic reformulations of nonlinear binary optimization problems," *Mathematical Programming*, vol. 162, pp. 115–144, 2017.
- [12] H. Dai, X. Chen, Y. Li, X. Gao, and L. Song, "A framework for differentiable discovery of graph algorithms," 2020.
- [13] T. Barrett, W. Clements, J. Foerster, and A. Lvovsky, "Exploratory combinatorial optimization with reinforcement learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 3243–3250.
- [14] H. Wang and P. Li, "Unsupervised learning for combinatorial optimization needs meta-learning," *arXiv preprint arXiv:2301.03116*, 2023.
- [15] D. Zhang, H. Dai, N. Malkin, A. Courville, Y. Bengio, and L. Pan, "Let the flows tell: Solving graph combinatorial problems with gflownets," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [16] S. Sanokowski, W. F. Berghammer, S. Hochreiter, and S. Lehner, "Variational annealing on graphs for combinatorial optimization," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [17] H. Sun, K. Goshvadi, A. Nova, D. Schuurmans, and H. Dai, "Revisiting sampling for combinatorial optimization," in *International Conference on Machine Learning*, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:260957235>
- [18] H. Sun, H. Dai, B. Dai, H. Zhou, and D. Schuurmans, "Discrete langevin sampler via wasserstein gradient flow," in *International Conference on Artificial Intelligence and Statistics*, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:250144582>
- [19] F. Glover, G. Kochenberger, and Y. Du, "A tutorial on formulating and using qubo models," *arXiv preprint arXiv:1811.11538*, 2018.
- [20] M. J. A. Schuetz, J. K. Brubaker, and H. G. Katzgraber, "Combinatorial optimization with physics-inspired graph neural networks," *Nature Machine Intelligence*, vol. 4, pp. 367 – 377, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:235727739>
- [21] M. J. A. Schuetz, J. K. Brubaker, Z. H. Zhu, and H. G. Katzgraber, "Graph coloring with physics-inspired graph neural networks," *ArXiv*, vol. abs/2202.01606, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:246485320>
- [22] I. Drori, A. Kharkar, W. R. Sickinger, B. Kates, Q. Ma, S. Ge, E. Dolev, B. L. Dietrich, D. P. Williamson, and M. Udell, "Learning to solve combinatorial optimization problems on real-world graphs in linear time," *2020 19th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pp. 19–24, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:219530437>
- [23] L. Gurobi Optimization, "Gurobi optimizer reference manual," 2023.
- [24] C. Bliklú, P. Bonami, and A. Lodi, "Solving mixed-integer quadratic programming problems with ibm-cplex: a progress report," in *Proceedings of the twenty-sixth RAMP symposium*, 2014, pp. 16–17.
- [25] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, and X. Sun, "Measuring and relieving the over-smoothing problem for graph neural networks from the topological view," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 3438–3445.
- [26] C. Cai and Y. Wang, "A note on over-smoothing for graph neural networks," *arXiv preprint arXiv:2006.13318*, 2020.
- [27] T. K. Rusch, M. M. Bronstein, and S. Mishra, "A survey on oversmoothing in graph neural networks," *arXiv preprint arXiv:2303.10993*, 2023.
- [28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [29] S. Yun, M. Jeong, R. Kim, J. Kang, and H. J. Kim, "Graph transformer networks," *Advances in neural information processing systems*, vol. 32, 2019.
- [30] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a general, powerful, scalable graph transformer," *Advances in Neural Information Processing Systems*, vol. 35, pp. 14 501–14 515, 2022.
- [31] Z. Wu, P. Jain, M. Wright, A. Mirhoseini, J. E. Gonzalez, and I. Stoica, "Representing long-range context for graph neural networks with global attention," *Advances in Neural Information Processing Systems*, vol. 34, pp. 13 266–13 279, 2021.
- [32] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski et al., "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [33] L. Brusca, L. C. Quaedvlieg, S. Skoulakis, G. G. Chrysos, and V. Cevher, "Maximum independent set: Self-training through dynamic programming," *arXiv preprint arXiv:2310.18672*, 2023.
- [34] Y. Min, F. Wenkel, M. Perlmutter, and G. Wolf, "Can hybrid geometric scattering networks help solve the maximum clique problem?" *Advances in Neural Information Processing Systems*, vol. 35, pp. 22 713–22 724, 2022.
- [35] M. X. Goemans and D. P. Williamson, "Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming," *Journal of the ACM (JACM)*, vol. 42, no. 6, pp. 1115–1145, 1995.
- [36] J. Kleinberg and E. Tardos, *Algorithm design*. Pearson Education India, 2006.
- [37] C. H. Papadimitriou and K. Steiglitz, *Combinatorial optimization: algorithms and complexity*. Courier Corporation, 1998.
- [38] N. C. Wormald, "Differential equations for random processes and random graphs," *The annals of applied probability*, pp. 1217–1235, 1995.
- [39] G. Bilbro, R. Mann, T. Miller, W. Snyder, D. van den Bout, and M. White, "Optimization by mean field annealing," *Advances in neural information processing systems*, vol. 1, 1988.
- [40] E. Khalil, H. Dai, Y. Zhang, B. Dilkina, and L. Song, "Learning combinatorial optimization algorithms over graphs," *Advances in neural information processing systems*, vol. 30, 2017.
- [41] N. Karalias and A. Loukas, "Erdos goes neural: an unsupervised learning framework for combinatorial optimization on graphs," *Advances in Neural Information Processing Systems*, vol. 33, pp. 6659–6672, 2020.
- [42] H. Sun, E. K. Guha, and H. Dai, "Annealed training for combinatorial optimization on graphs," *arXiv preprint arXiv:2207.11542*, 2022.
- [43] J. Toenshoff, M. Ritzert, H. Wolf, and M. Grohe, "Graph neural networks for maximum constraint satisfaction," *Frontiers in artificial intelligence*, vol. 3, p. 580607, 2021.
- [44] K. Xu and W. Li, "Exact phase transitions in random constraint satisfaction problems," *Journal of Artificial Intelligence Research*, vol. 12, pp. 93–103, 2000.
- [45] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *science*, vol. 286, no. 5439, pp. 509–512, 1999.
- [46] C. Morris, N. M. Kriege, F. Bause, K. Kersting, P. Mutzel, and M. Neumann, "Tudataset: A collection of benchmark datasets for learning with graphs," *arXiv preprint arXiv:2007.08663*, 2020.